BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

LESSON 054 · Lesson 054 · Hash Tables and Dictionary<TKey,TValue>

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. Which data structure uses FIFO in BFS?

Q2. What does a dictionary map?

Q3. What is a collision in hashing?

Q4. 023

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

hash function — Maps keys to hash values.

collision — Different keys share a target bucket.

bucket — Hash table storage location.

load factor — Filled entries relative to capacity.

New learning

Discover — A hash function maps keys to hash values that guide placement/lookups. A collision occurs if distinct keys map to the same location/bucket.

Trace — A .NET Dictionary<TKey,TValue> handles collision management internally; do not equate its actual buckets with simplistic modulo examples.

Evaluate — To read a missing key safely, use TryGetValue. Hash-table operations are often expected O(1) but poor collisions can degrade performance.

Worked example

Apply a toy hash h(k)=k mod 5 to keys 7 and 12. What happens?

1. Explain the trace — Both keys collide in bucket 2; a table must resolve collision by chaining or probing.

We Do · Practise with guidance

Apply the same toy hash h(k)=k mod 5 to keys 8 and 11. Give buckets.

```
h(8)=3
h(11)=1
```

Explain the trace

No collision: key 8 maps bucket 3, key11 maps bucket1.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

With toy h(k)=k mod5, find buckets for keys 8,13,19. [6]

Question 2

Write C# Dictionary<string,int> code storing marks Ada80 and Ben65, then safely test whether Cleo exists. [6]

Question 3

Compare chaining and linear probing for toy hash collisions, and state two performance concerns. [6]

Exit Ticket · Answer before checking

Hash tables can never have collisions when keys are different.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. A queue.

Q2. Keys to associated values.

Q3. Different keys map to same hash location.

Q4. With toy h(k)=k mod7, hash keys 9,16,22. Which collide?

You Do pages · Independent questions

Q1. 8→3 [2],13→3 [2],19→4 [1], 8/13 collide [1].

Q2. var d=new Dictionary<string,int>{{"Ada",80},{"Ben",65}}; [3]; if(d.TryGetValue("Cleo",out int mark)) ... [3], handles missing key.

Q3. Chaining keeps multiple entries per bucket [2]; linear probing searches next available slot [2]; load factor and clustering affect lookup [2].

Exit Ticket · Model answer

False. Finite bucket spaces mean collisions may occur.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed: